

Práctica 3 – Comunicación Cliente–Servidor usando Sockets en C++

Grado en Tecnologías Digitales para la Empresa
Asignatura: Sistemas Operativos

Dr. Jaime Andrés Rincón Arango

Curso 2025-2026

Introducción a los sockets

Un **socket** es un mecanismo de comunicación que permite el intercambio de datos entre procesos, ya sea dentro del mismo sistema o entre distintos equipos conectados a una red. En aplicaciones distribuidas, los sockets constituyen la base del modelo **cliente–servidor**, donde un programa cliente establece una conexión con un programa servidor utilizando una dirección IP y un número de puerto. A través de esta conexión se pueden enviar y recibir datos en tiempo real. En esta práctica se utilizarán **sockets TCP/IP**, que permiten una comunicación fiable entre aplicaciones. El protocolo TCP garantiza que los datos lleguen correctamente, en orden y sin pérdidas.

El uso de sockets es fundamental en numerosos sistemas actuales, como:

- aplicaciones cliente–servidor
- sistemas IoT industriales
- monitorización remota de sensores
- transmisión de datos en tiempo real
- sistemas distribuidos

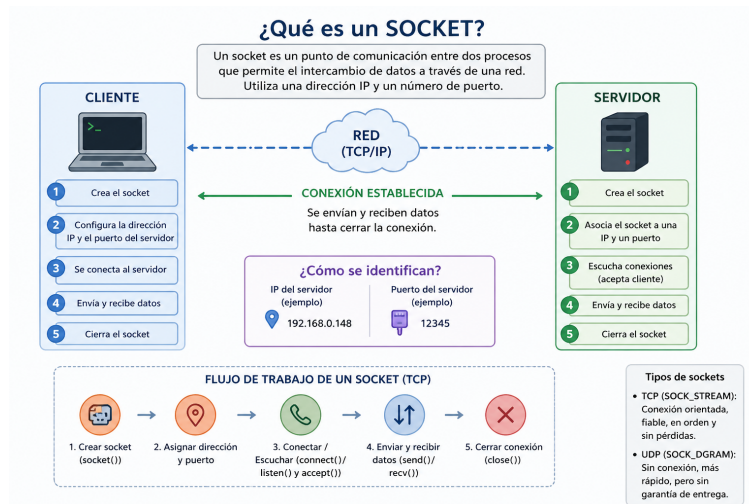


Figura 1: Que es un Socket.

Contexto de la práctica

En entornos industriales modernos (Industria 4.0), los sistemas de monitorización remota permiten recoger información procedente de sensores distribuidos en planta mediante dispositivos Edge como Raspberry Pi. En esta práctica se simula un escenario real de supervisión industrial donde un servidor transmite datos de un sensor de temperatura a un cliente desarrollado en C++ mediante sockets TCP/IP. Este tipo de arquitectura es habitual en:

- mantenimiento predictivo
- control térmico de maquinaria
- redes IoT industriales

El objetivo será desarrollar un cliente capaz de conectarse al servidor, recibir datos en tiempo real y almacenarlos correctamente en un archivo local.

Servidor proporcionado en los materiales de la práctica

Dentro de los materiales asociados a esta práctica se proporcionará un servidor desarrollado en Python. Este servidor simula el comportamiento de una cámara térmica industrial que envía datos de temperatura al cliente a través de una conexión TCP/IP.

El estudiante deberá ejecutar previamente este servidor Python antes de lanzar el programa cliente en C++, ya que será el encargado de enviar los datos que posteriormente deberán ser recibidos, almacenados y procesados por el cliente.

Este servidor simula el funcionamiento típico de un dispositivo Edge en un entorno de monitorización industrial, donde sensores térmicos transmiten información en tiempo real a sistemas remotos para su almacenamiento y análisis.

La dirección IP y el puerto necesarios para la conexión serán indicados dentro de los materiales de la práctica.



Figura 2: Flujo de la Practica.

Código base proporcionado

Para el desarrollo de esta práctica se proporcionará al estudiante un código base en C++. Este código contiene la estructura inicial para crear un socket TCP/IP, configurar la dirección del servidor, establecer la

conexión y recibir datos mediante la función `recv()`.

El código base deberá utilizarse obligatoriamente como punto de partida para realizar la conexión con un servidor desarrollado en Python, el cual simula el envío de datos procedentes de una cámara o sensor remoto.

El estudiante deberá corregir, completar, modularizar y adaptar este código para que funcione correctamente siguiendo los requisitos de la práctica.

```
#include <iostream>
#include <vector>
#include <cstring>
#include <arpa/inet.h>
#include <unistd.h>
#include <fstream>

int main() {
    // Crear un socket TCP/IP
    int sockfd = socket(AF_INET, SOCK_STREAM, 0);

    if (sockfd == -1) {
        std::cerr << "Error al crear el socket" << std::endl;
        return 1;
    }

    // Conectar el socket al servidor
    struct sockaddr_in serv_addr;
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(12345);

    if (inet_pton(AF_INET, "192.168.0.148", &serv_addr.sin_addr) <= 0) {
        std::cerr << "Direccion invalida o no soportada" << std::endl;
        return 1;
    }

    if (connect(sockfd, (struct sockaddr *)&serv_addr,
                sizeof(serv_addr)) == -1) {
        std::cerr << "Error al conectar" << std::endl;
        return 1;
    }

    // Recibir los datos como bytes
    std::vector<float> floats;
    float buffer;

    while (recv(sockfd, &buffer, sizeof(buffer), 0) > 0) {
        floats.push_back(buffer);
    }

    // Guardar los datos en un archivo de texto
    // Implementar

    // Imprimir los numeros de punto flotante
    std::cout << "Numeros de punto flotante recibidos:" << std::endl;

    for (float f : floats) {
        std::cout << f << std::endl;
    }
}
```

```
// Cerrar el socket
close(sockfd);

return 0;
}
```

Importante: el código anterior no debe entregarse sin modificar. El estudiante deberá transformarlo en una solución completa utilizando funciones, incluyendo como mínimo:

- una función para crear el socket
- una función para configurar la dirección del servidor
- una función para conectar con el servidor
- una función para recibir los datos
- una función para guardar los datos en archivo
- una función para imprimir los resultados

El servidor en Python simulará el comportamiento de una cámara o dispositivo remoto que envía datos al cliente. Por tanto, el cliente C++ deberá conectarse correctamente a dicho servidor y procesar los datos recibidos.

Objetivos

Los objetivos principales de la práctica son:

- Comprender el modelo cliente-servidor
- Establecer comunicaciones mediante sockets TCP/IP
- Implementar programas de comunicación en C++
- Recibir datos desde sensores remotos
- Almacenar información en archivos
- Implementar control básico de errores en comunicaciones

Arquitectura del sistema

El flujo de datos será:

Sensor temperatura → Servidor → Cliente C++ → Archivo local

El servidor enviará valores tipo `float` correspondientes a lecturas de temperatura.

El cliente deberá:

- conectarse al servidor
- recibir los datos
- almacenarlos en memoria
- guardarlos en archivo local

Tareas a desarrollar

El estudiante deberá desarrollar un programa cliente en C++ capaz de comunicarse con un servidor mediante sockets TCP/IP para recibir datos procedentes de una cámara térmica simulada en Python.

El programa deberá:

- crear y configurar correctamente un socket TCP/IP
- establecer la conexión con el servidor proporcionado
- recibir los datos enviados por el servidor mediante `recv()`
- almacenar los datos recibidos en estructuras dinámicas (arrays o vectores)
- guardar los datos en un archivo de salida
- mostrar los resultados por pantalla
- implementar control básico de errores en la comunicación
- estructurar el programa utilizando funciones propias

Entregables

El estudiante deberá entregar:

- archivo fuente `cliente.cpp`
- archivo generado con los datos recibidos
- script en Bash para ejecutar el programa mediante `sh`
- capturas de ejecución del programa
- informe técnico en PDF
- repositorio Git correctamente organizado

Criterios de evaluación

Puntuación total: 10 puntos

Informe técnico (3 puntos)

El informe deberá incluir obligatoriamente:

- Portada
- Introducción y contextualización del problema
- Explicación del funcionamiento del programa desarrollado
- Explicación del uso de arrays y matrices
- Explicación de la lectura del archivo de entrada
- Explicación de la escritura del archivo de salida
- Explicación del script en Bash para la ejecución mediante `sh`
- Capturas de pantalla de la ejecución del programa

- Interpretación de los resultados obtenidos
- Conclusiones técnicas

La ausencia de apartados supondrá penalización proporcional.

Implementación del socket (2 puntos)

- creación correcta del socket
- configuración IP y puerto
- conexión funcional con el servidor

Recepción y almacenamiento de datos (2 puntos)

- uso correcto de `recv()`
- almacenamiento correcto en arrays o vectores

Escritura del archivo de salida (1.5 puntos)

- uso correcto de `ofstream`
- generación correcta del archivo

Modularización mediante funciones (1 punto)

- correcta estructuración del programa
- separación lógica en funciones propias

Repositorio Git (0.5 puntos)

Debe contener:

- README
- código organizado
- capturas de ejecución

Fecha de entrega

La práctica deberá entregarse antes del:

10 Mayo 2026

